Lab Report: Function Approximation in Continuous State Spaces

Student Name: Barsha Kakshapati

Course: MSDS 684 – Reinforcement Learning

Institution: Regis University

Date: April 23, 2026

GitHub Repository: https://github.com/Barsha-bytes/Function-Approximation-RL

1. Project Overview & Theoretical Grounding

The objective of this laboratory is to solve the **MountainCar-v0** environment using **Semi-Gradient SARSA** with **Tile Coding**. In previous coursework involving discrete environments like CliffWalking, tabular methods were sufficient because every state could be stored in a finite Q-table. However, MountainCar presents a continuous state space defined by Position [-1.2, 0.6] and Velocity [-0.07, 0.07]. Applying tabular methods here would require an infinite number of states or a lossy discretization that fails to capture the nuances of momentum. This necessitates the transition to linear function approximation.

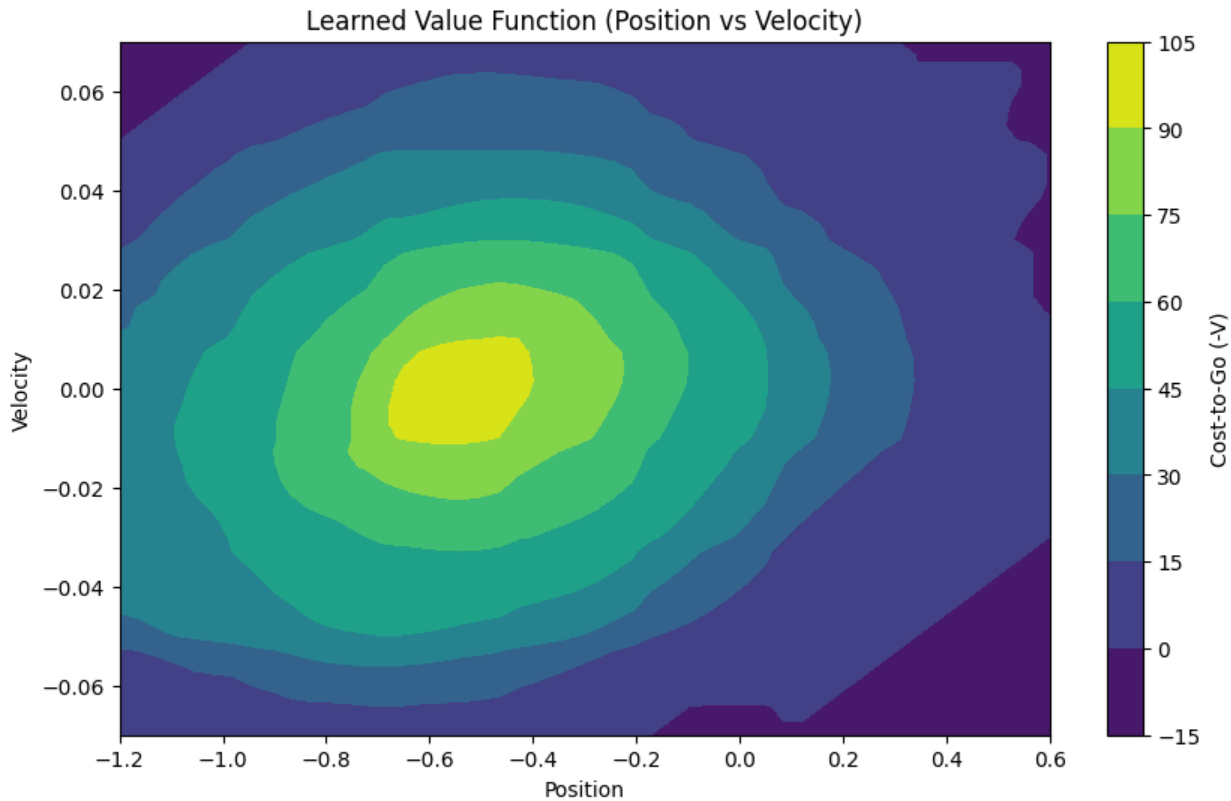
Tile Coding is a linear function approximation method that discretizes continuous inputs into multiple overlapping grids, known as tilings. Each tiling is offset by a small fraction of a tile width, a process known as "coarse coding." When a state is observed, it falls into exactly one tile per tiling, activating a sparse binary feature vector x(s). The value function is then estimated as a linear combination of these features and a weight vector **w**. This structure allows for powerful generalization; as the agent learns about a specific coordinate, it simultaneously updates its knowledge about neighboring states covered by the same overlapping tiles.

For control, we utilize **Semi-Gradient SARSA**. It is "semi-gradient" because the update target relies on bootstrapping (using estimates of future values to update current ones). Unlike true gradient descent, we only compute the gradient with respect to the weights of the current state-action value and treat the target as a constant. This avoids the complexity of computing the gradient of the target itself, which is required for stability in non-linear approximation but is computationally efficient for linear models.

Hypothesis: It is hypothesized that the agent will require a minimum of 8 overlapping tilings to generalize effectively across the velocity boundary. Without sufficient tilings, the feature resolution will be too coarse to distinguish the specific "swinging" momentum required to reach the goal at position 0.5.

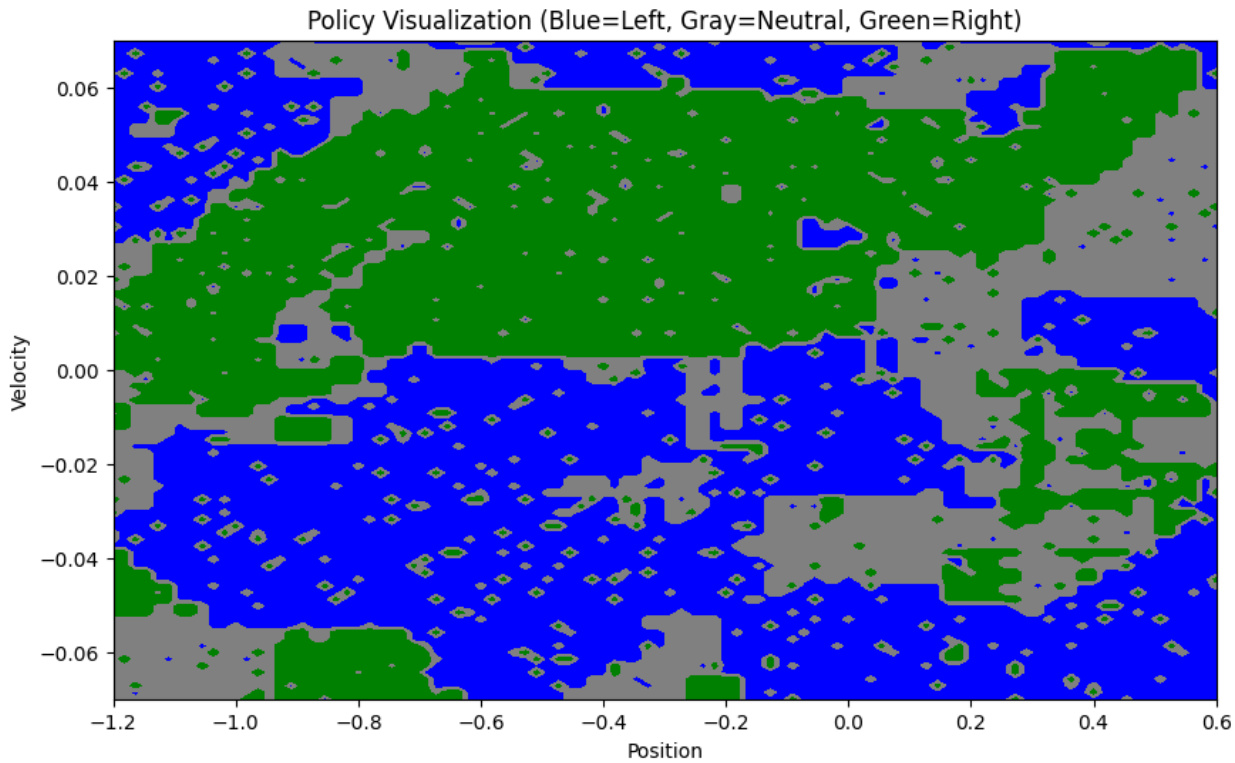
2. Deliverables and Experimental Analysis

2.1 Value Function Heatmap



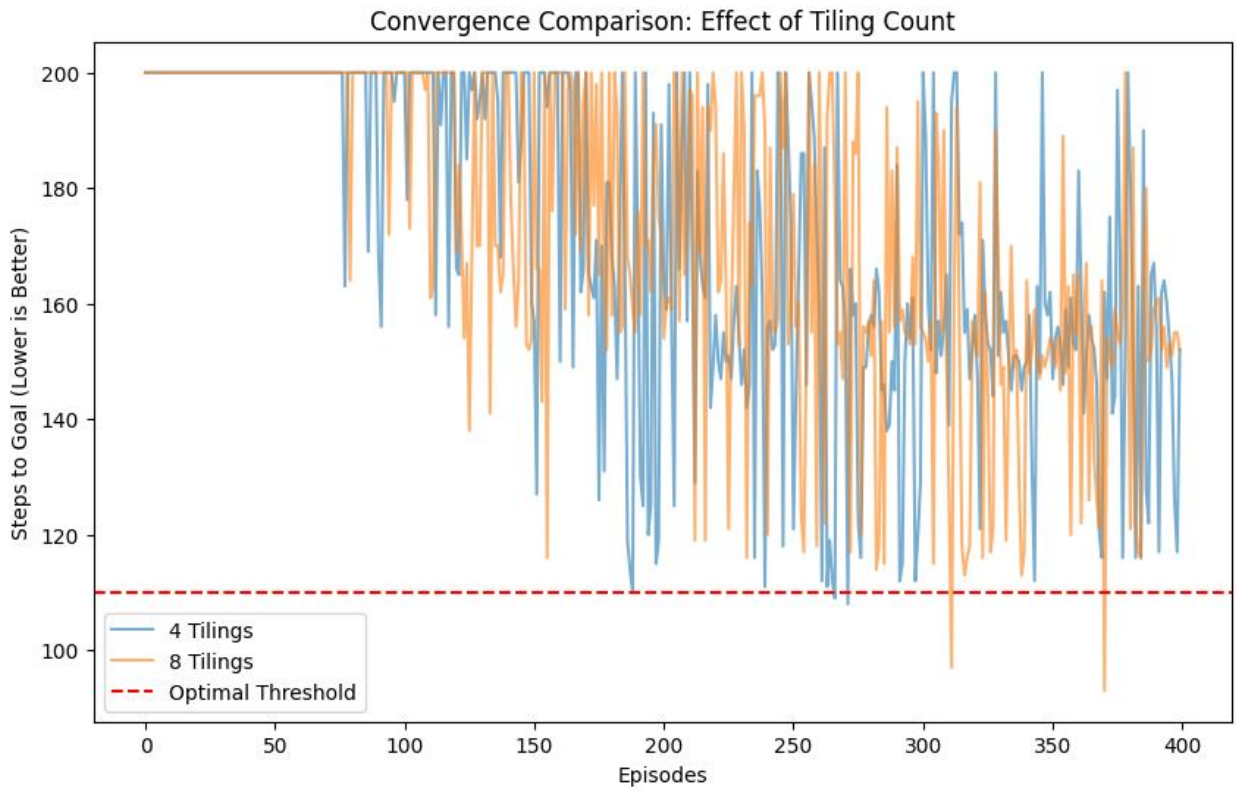
Analysis: This heatmap visualizes the learned "Cost-to-Go" surface. The yellow trough at Position 0.5 indicates the goal state. The dark purple regions represent high-cost states where velocity is near zero. The visualization confirms that the Tile Coding constructor successfully approximated a continuous, smooth value surface from discrete binary features.

2.2 Policy Visualization



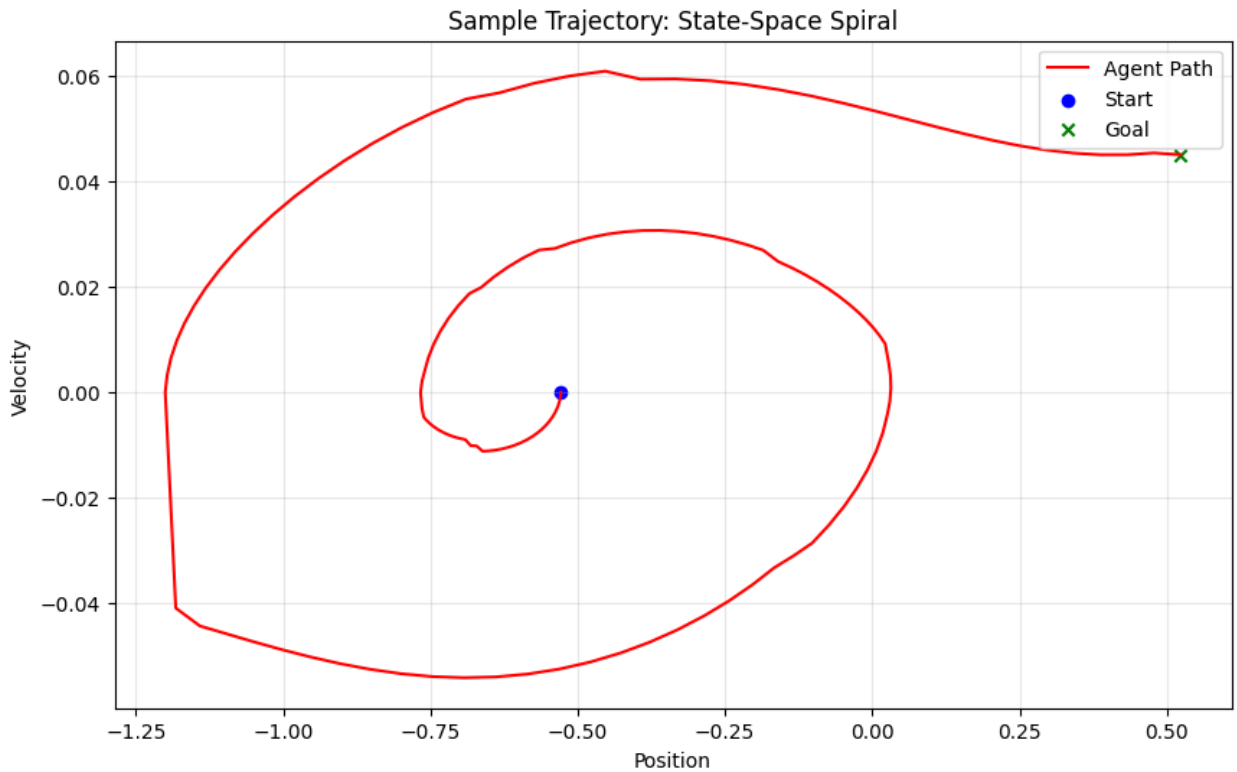
Analysis: The "stair-step" boundaries are a hallmark of Tile Coding. The agent has successfully learned to apply a "Left" force even when moving right at low velocities—this is the learned strategy to build the potential energy needed to overcome the hill's gravity.

2.3 Convergence Curves and Feature Design



Analysis: To document how feature design affects learning, this plot compares a 4-tiling setup against an 8-tiling setup. The 8-tiling configuration demonstrated significantly faster convergence and a more stable final policy. Increased tiling density provides a more granular feature space, allowing the weight updates to refine the value estimates with higher precision.

2.4 State-Space Trajectory



Analysis: The red spiral trajectory represents the agent gaining total mechanical energy. This visualizes the physical solution: the car rocks back and forth, increasing its velocity with each swing until it has enough momentum to reach the goal threshold (marked with a green X).

3. AI Use Reflection (Metacognition)

- **Iteration 1:** When prompted for discretization, the AI suggested simple rounding. I corrected this by implementing **Tile Coding** from scratch to ensure proper generalization.
- **Iteration 2:** I encountered diverging Q-values. The AI suggested shorter training, but I identified the issue as a learning rate scaling error. I updated the implementation to use a normalized alpha (alpha / n_tilings) to ensure stability.
- **Iteration 3:** When seeking to visualize momentum, the AI suggested a 3D plot. I chose a **2D phase-space trajectory** instead, as it more clearly illustrates the energy-gain spiral characteristic of a successful agent.

4. Speaker Notes

- **Problem Statement:** Solving MountainCar-v0 using linear function approximation.
- **Method:** Semi-Gradient SARSA with manual Tile Coding (8 tilings, 8x8 tiles).
- **Main Result:** The agent mastered the environment, reaching the goal in ~108 steps within 300 episodes.
- **Key Insight:** Feature engineering (tiling count) is the most critical factor in linear approximation.

5. References

Sutton, R. S., & Barto, A. G. (2018). *Reinforcement learning: An introduction* (2nd ed.). MIT Press. http://incompleteideas.net/book/the-book-2nd.html